```
>>> from bands.models import Musician
>>> lennon = Musician.objects.get(last_name="Lennon")
>>> lennon
<Musician: Musician(id=2, last_name=Lennon)>
```

Now that you have a reference to a musician and a band, you can associate the two together. The `Band` class has a query manager named after the `ManyToManyField`: `musicians`. The `.add()` method on the `.musicians` query manager adds a `Musician` to the `Band`:

```
>>> beatles.musicians.add(lennon)
```

As `.musicians` is a query manager, you can run queries on it. Running `.all()` returns all the `Musician` objects currently associated with the `Band`. In this case, that's only Mr. Lennon:

```
>>> beatles.musicians.all()
<QuerySet [<Musician: Musician(id=2, last_name=Lennon)>]>
```

But wait, there's more! Yep, still more—sort of the same kind of more. You can also go backward. The `Musician` class now has a query manager named `band_set`. Using this query manager, you can determine which bands a given musician belongs to:

```
>>> lennon.band_set.all()
<QuerySet [<Band: Band(id=1, name=The Beatles)>]>
```

Because this is a many-to-many relationship, musicians can be associated with multiple bands. Let's create a supergroup trio band, named *Wishful Thinking*, and add all three of our musicians:

```
>>> vai = Musician.objects.get(last_name="Vai")
>>> bonham = Musician.objects.get(last_name="Bonham")
>>> wishful = Band.objects.create(name="Wishful Thinking")
>>> wishful.musicians.add(lennon, vai, bonham)
>>> wishful.musicians.all()
<QuerySet [<Musician: Musician(id=1, last_name=Vai)>,
➥ <Musician: Musician(id=2, last_name=Lennon)>,
➥ <Musician: Musician(id=3, last_name=Bonham)>]>
```

Recall that with the `ForeignKey` field, you need to be explicit about the behavior when a related object gets deleted. Deleting a `Venue` without deleting the corresponding `Room` objects doesn't make a lot of sense, so you `CASCADE`. With a many-to-many relationship, this problem doesn't exist. If you wanted to delete Mr. Lennon, Django takes care of removing any associated `Band` relationships. Note that this does not remove the `Band`. If you delete all the members of Wishful Thinking, you will still have a `Wishful ThinkingBand` object—it just won't have any related `Musicians`. If you are trying to remove all the musicians and their associate band, you need to make separate explicit delete calls on both object types.